

JAVA

# Bancos de Dados com JDBC e JPA

Conectando, consultando e persistindo dados: de Statement a Entity, na prática

Apostila completa sobre acesso a bancos de dados em Java: fundamentos de SQL, conexão via JDBC, consultas parametrizadas e stored procedures, além da camada de abstração oferecida pelo JPA — com exemplos aplicados ao sistema de agendamento de uma clínica veterinária.

---

## CONTEÚDO DA APOSTILA

- |                                                               |                                                                |
|---------------------------------------------------------------|----------------------------------------------------------------|
| <b>01</b> Bancos de Dados e o Papel do JDBC                   | <b>13</b> Execução em Lote com executeBatch                    |
| <b>02</b> Fundamentos de um Banco de Dados Relacional         | <b>14</b> Update, Delete e Inserção de Registros Relacionados  |
| <b>03</b> SQL: DDL, DML e o Acrônimo CRUD                     | <b>15</b> Stored Procedures e o CallableStatement              |
| <b>04</b> Relacionamentos, Normalização e Joins               | <b>16</b> Parâmetros IN, OUT e INOUT                           |
| <b>05</b> Instalando o MySQL e Modelando com o Workbench      | <b>17</b> Stored Functions e Escape Sequences do JDBC          |
| <b>06</b> O que é JDBC e o Papel do Driver                    | <b>18</b> Introdução ao JPA e ao ORM                           |
| <b>07</b> Conectando-se ao Banco com JDBC                     | <b>19</b> Entity, EntityManager e Persistence Context          |
| <b>08</b> Statement e ResultSet: Executando e Lendo Consultas | <b>20</b> Consultas com JPQL                                   |
| <b>09</b> SQL Injection e o Padrão ANSI SQL                   | <b>21</b> Joins em JPQL e o JPA Tuple                          |
| <b>10</b> PreparedStatement: Consultas Parametrizadas         | <b>22</b> CriteriaBuilder: Consultas Programáticas e Type-Safe |
| <b>11</b> Vantagens do PreparedStatement Sobre o Statement    | <b>23</b> Native Queries e o Panorama Final das Consultas JPA  |
| <b>12</b> Transações: AUTOCOMMIT, Commit e Rollback           |                                                                |

## Sumário

### 01 Bancos de Dados e o Papel do JDBC

Por que uma aplicação Java precisa conversar com um banco de dados, e quem faz essa ponte

### 02 Fundamentos de um Banco de Dados Relacional

Schema, tabela, índice, view e usuário — o vocabulário básico de um RDBMS

### 03 SQL: DDL, DML e o Acrônimo CRUD

A linguagem que usamos para descrever, popular e consultar um banco de dados

### 04 Relacionamentos, Normalização e Joins

Como tabelas separadas se conectam, e como voltar a juntá-las em uma consulta

### 05 Instalando o MySQL e Modelando com o Workbench

Colocando o banco de dados de pé e desenhando o diagrama de entidades da PataViva

### 06 O que é JDBC e o Papel do Driver

A API que padroniza o acesso a bancos de dados a partir de uma aplicação Java

### 07 Conectando-se ao Banco com JDBC

Montando a connection string certa para cada fabricante e abrindo a primeira conexão

### 08 Statement e ResultSet: Executando e Lendo Consultas

O jeito mais direto de mandar SQL para o banco e percorrer o que ele devolve

### 09 SQL Injection e o Padrão ANSI SQL

Como uma entrada de usuário mal tratada vira uma falha de segurança, e por que padronizar o SQL importa

### 10 PreparedStatement: Consultas Parametrizadas

Pré-compilando uma instrução SQL e substituindo valores por marcadores seguros

### 11 Vantagens do PreparedStatement Sobre o Statement

Seis motivos práticos para preferir instruções parametrizadas no dia a dia

### 12 Transações: AUTOCOMMIT, Commit e Rollback

Garantindo que um grupo de operações relacionadas seja tratado como uma coisa só

### 13 Execução em Lote com executeBatch

Agrupando várias instruções em uma única viagem até o banco de dados

### 14 Update, Delete e Inserção de Registros Relacionados

Modificando dados existentes sem quebrar a integridade referencial entre tabelas

### 15 Stored Procedures e o CallableStatement

Chamando rotinas que já vivem compiladas dentro do próprio banco de dados

### 16 Parâmetros IN, OUT e INOUT

Os três jeitos de trocar dados com uma stored procedure através do CallableStatement

### 17 Stored Functions e Escape Sequences do JDBC

A outra metade das rotinas armazenadas, e a sintaxe que o JDBC usa para ficar portátil entre bancos

### 18 Introdução ao JPA e ao ORM

Uma camada de abstração entre o código Java e o SQL que ele geraria manualmente

### 19 Entity, EntityManager e Persistence Context

Quem gerencia o ciclo de vida de um objeto mapeado, e onde ele vive enquanto isso

## 20 Consultas com JPQL

Uma linguagem de consulta orientada a objetos, pensada para trabalhar com entidades, não com tabelas

## 21 Joins em JPQL e o JPA Tuple

Consultando dados que atravessam mais de uma entidade relacionada, sem trazer o objeto inteiro

## 22 CriteriaBuilder: Consultas Programáticas e Type-Safe

Montando uma consulta com objetos Java em vez de uma string de JPQL

## 23 Native Queries e o Panorama Final das Consultas JPA

Recorrendo ao SQL puro quando necessário, e comparando as três formas de consultar com JPA

# Bancos de Dados e o Papel do JDBC

*Por que uma aplicação Java precisa conversar com um banco de dados, e quem faz essa ponte*

---

Praticamente toda aplicação séria precisa guardar dados em algum lugar que sobreviva ao encerramento do programa. Uma lista em memória desaparece assim que a JVM é encerrada, então a solução mais comum, há décadas, é persistir esses dados em um banco de dados relacional — um software especializado em armazenar registros de forma organizada, consultável e durável.

O problema é que cada fabricante de banco de dados — MySQL, PostgreSQL, Oracle, SQL Server e outros — tem seu próprio protocolo de comunicação interno. Se a linguagem Java exigisse um código diferente para cada um desses protocolos, qualquer aplicação que precisasse trocar de banco teria que ser reescrita quase por completo. É exatamente esse problema que o JDBC (Java Database Connectivity) resolve.

Ao longo desta apostila, vamos construir gradualmente o sistema de agendamento de uma clínica veterinária fictícia, a PataViva. Ela cadastra tutores, seus pets, os veterinários do quadro clínico e as consultas agendadas entre eles — um cenário simples, mas rico o bastante para explorar tabelas relacionadas, chaves estrangeiras e junções, que são o pão de cada dia de quem trabalha com bancos de dados relacionais.

## O que este material cobre

- ▶ Como estabelecer uma conexão com um banco de dados através de uma connection string e de um driver JDBC.
- ▶ Como consultar dados usando SQL (Structured Query Language) e processar os resultados retornados.
- ▶ Como executar instruções parametrizadas com PreparedStatement, tornando o SQL reutilizável, mais flexível e mais seguro.
- ▶ Como chamar rotinas armazenadas no próprio banco de dados com CallableStatement.
- ▶ Como uma camada de mapeamento objeto-relacional (ORM), através do JPA, pode eliminar boa parte do código repetitivo visto nos capítulos de JDBC.

### DICA — Por que MySQL nesta apostila

O MySQL foi escolhido como banco de referência por ser gratuito, ter uma instalação simples e — o mais importante para os capítulos sobre CallableStatement — suportar procedures e functions armazenadas nativamente.

# Fundamentos de um Banco de Dados Relacional

*Schema, tabela, índice, view e usuário — o vocabulário básico de um RDBMS*

No nível mais alto de organização de um Sistema de Gerenciamento de Banco de Dados Relacional (RDBMS), os dados vivem dentro de um schema: um agrupamento lógico, um namespace, que reúne objetos relacionados como tabelas, views e procedures. É comum pensar no schema como uma gaveta que organiza tudo o que pertence a uma mesma aplicação.

Vale um esclarecimento importante: o termo schema não significa exatamente a mesma coisa em todo banco de dados. Em sistemas como PostgreSQL e Oracle, um schema é um namespace dentro de um banco — um único banco pode conter vários schemas. Já no MySQL, schema e banco de dados são tratados como sinônimos: você cria um database, mas o MySQL Workbench frequentemente se refere a ele como schema na interface.

## Objetos comuns de um banco de dados

| Objeto  | Descrição                                                                                                                           |
|---------|-------------------------------------------------------------------------------------------------------------------------------------|
| Tabela  | Armazena registros (linhas) organizados em campos (colunas), com valores específicos para cada registro.                            |
| Índice  | Estrutura auxiliar com o nome da tabela, um valor de chave e um localizador de registro, usada para acelerar a busca por uma linha. |
| View    | Uma consulta armazenada, acessível como se fosse uma tabela, que esconde do cliente os detalhes de implementação.                   |
| Usuário | Representa um conjunto de privilégios de acesso concedidos a uma conta sobre os objetos do banco.                                   |

Para a PataViva, o schema pataviva vai reunir as tabelas tutor, pet, veterinario e consulta. Uma chave primária identifica de forma única cada registro dessas tabelas — por exemplo, cada tutor tem um id que nenhum outro tutor compartilha, e é através desse identificador que outras tabelas vão referenciá-lo.

### REGRA — Chave primária é identidade, não descrição

Um índice acelera buscas; a chave primária, especificamente, garante que cada linha de uma tabela tenha um identificador único e estável, que outras tabelas podem usar para apontar de volta para ela.

# SQL: DDL, DML e o Acrônimo CRUD

*A linguagem que usamos para descrever, popular e consultar um banco de dados*

Assim como qualquer software tem uma linguagem própria para ser operado, um banco de dados relacional é operado através da Structured Query Language, o SQL. É essa linguagem que permite criar objetos, popular tabelas com informação, estabelecer relacionamentos e consultar dados. Para bancos relacionais, o SQL foi padronizado pela ANSI, ainda que cada fabricante mantenha algumas cláusulas próprias — voltaremos a esse detalhe no capítulo 9.

O SQL se divide em duas grandes categorias, com propósitos bem distintos. A Data Definition Language (DDL) define, cria e modifica a estrutura dos objetos do banco — ela nunca manipula os dados armazenados, apenas as estruturas que os organizam.

| Comando DDL | Descrição                                                           |
|-------------|---------------------------------------------------------------------|
| CREATE      | Cria objetos do banco, como tabelas, índices, views e schemas.      |
| ALTER       | Modifica a estrutura de um objeto já existente.                     |
| DROP        | Remove um objeto do banco por completo.                             |
| TRUNCATE    | Remove todas as linhas de uma tabela, mantendo a estrutura intacta. |
| RENAME      | Renomeia um objeto do banco.                                        |

Já a Data Manipulation Language (DML) é usada para interagir com os dados propriamente ditos, armazenados dentro desses objetos — inserir, atualizar, buscar e apagar registros.

| Comando DML | Descrição                                   |
|-------------|---------------------------------------------|
| SELECT      | Recupera dados de uma ou mais tabelas.      |
| INSERT      | Adiciona novas linhas a uma tabela.         |
| UPDATE      | Modifica dados já existentes em uma tabela. |
| DELETE      | Remove linhas de uma tabela.                |

Esses quatro comandos de DML costumam ser lembrados por outro acrônimo bastante conhecido: CRUD. Create equivale a um INSERT; Read equivale a um SELECT; Update é a própria palavra UPDATE; e Delete corresponde ao comando DELETE. O CRUD representa as quatro operações fundamentais de qualquer sistema que gerencia dados persistentes, e vamos reencontrar essas quatro letras mais adiante, quando falarmos sobre o EntityManager do JPA.

```
CREATE TABLE tutor (  
    id INT AUTO_INCREMENT PRIMARY KEY,  
    nome VARCHAR(80) NOT NULL,  
    telefone VARCHAR(20)  
);  
  
INSERT INTO tutor (nome, telefone) VALUES ('Marina Souza', '11 98888-2323');  
  
SELECT nome, telefone FROM tutor WHERE id = 1;
```

# Relacionamentos, Normalização e Joins

*Como tabelas separadas se conectam, e como voltar a juntá-las em uma consulta*

Diferente de uma planilha isolada, um RDBMS guarda dados em múltiplas tabelas, associadas entre si por diferentes tipos de relacionamento. Na PataViva, um tutor pode ter vários pets, mas cada pet pertence a um único tutor — uma relação um-para-muitos. Já uma consulta associa exatamente um pet a exatamente um veterinário em uma data específica.

| Relacionamento     | Descrição                                                                                                                  |
|--------------------|----------------------------------------------------------------------------------------------------------------------------|
| Um para um         | Uma linha da primeira tabela se relaciona com apenas uma linha da segunda.                                                 |
| Um para muitos     | Uma linha da primeira tabela se relaciona com várias linhas da segunda.                                                    |
| Muitos para muitos | Várias linhas da primeira tabela se relacionam com várias linhas da segunda, geralmente através de uma tabela associativa. |

O processo de organizar os dados de um banco para reduzir redundância e melhorar a integridade é chamado de normalização. Na prática, significa quebrar uma tabela grande em tabelas menores e relacionadas, seguindo um conjunto de regras conhecidas como formas normais, cujo objetivo é garantir que cada informação exista em um único lugar — evitando, por exemplo, repetir o telefone do tutor em toda linha de consulta que envolva um dos seus pets.

O efeito colateral dessa organização é que os dados ficam espalhados em silos separados, o que é ótimo para manutenção e armazenamento, mas complica a hora de buscar informações que cruzam várias tabelas. É para isso que existe a cláusula JOIN: ela combina linhas de duas ou mais tabelas, com base em um campo em comum, permitindo montar uma única consulta que devolve dados de várias tabelas relacionadas ao mesmo tempo.

| Tipo de Join | Resultado                                                                                    |
|--------------|----------------------------------------------------------------------------------------------|
| INNER JOIN   | Retorna apenas as linhas de ambas as tabelas em que a condição de junção é satisfeita.       |
| LEFT JOIN    | Retorna todas as linhas da tabela à esquerda, mesmo sem correspondência na tabela à direita. |
| RIGHT JOIN   | Retorna todas as linhas da tabela à direita, mesmo sem correspondência na tabela à esquerda. |
| FULL JOIN    | Retorna todas as linhas de ambas as tabelas, havendo ou não correspondência.                 |
| CROSS JOIN   | Retorna todas as combinações possíveis entre as linhas das duas tabelas.                     |



```
SELECT t.nome AS tutor, p.nome AS pet
FROM tutor t
INNER JOIN pet p ON p.tutor_id = t.id
WHERE t.nome = 'Marina Souza';
```

#### **DICA — Junte só o que precisa**

Um INNER JOIN é a escolha certa quando você só quer registros que realmente têm correspondência nos dois lados — por exemplo, tutores que já têm ao menos um pet cadastrado. Use LEFT JOIN quando quiser manter todos os tutores na resposta, mesmo os que ainda não cadastraram nenhum pet.

# Instalando o MySQL e Modelando com o Workbench

*Colocando o banco de dados de pé e desenhando o diagrama de entidades da PataViva*

Antes de escrever qualquer linha de Java, é preciso ter um banco de dados MySQL rodando localmente. O instalador varia por sistema operacional, mas o caminho de obtenção é sempre o mesmo: o site oficial do MySQL Community, mantido pela Oracle, disponibiliza pacotes gratuitos para Linux, macOS e Windows.

| Sistema Operacional | Forma de instalação recomendada                                    |
|---------------------|--------------------------------------------------------------------|
| Linux               | Repositório oficial (Yum, APT ou SUSE, dependendo da distribuição) |
| macOS               | Pacote MySQL Community Server (arquivo .dmg)                       |
| Windows             | MySQL Installer for Windows                                        |

Depois de instalado, o MySQL Workbench é a ferramenta gráfica oficial para administrar o banco — criar schemas, executar consultas ad-hoc e, o que nos interessa agora, desenhar diagramas. Um Diagrama de Entidade-Relacionamento (ERD) é parecido com um diagrama de classes: cada entidade representa uma tabela, e as linhas conectando as entidades descrevem como elas se relacionam.

Para a PataViva, o ERD tem quatro entidades. Tutor se relaciona um-para-muitos com Pet; Veterinário se relaciona um-para-muitos com Consulta; e Pet também se relaciona um-para-muitos com Consulta, já que um mesmo pet pode ter várias consultas ao longo do tempo, mas cada consulta pertence a exatamente um pet e a exatamente um veterinário.

```
CREATE DATABASE pataviva;

USE pataviva;

CREATE TABLE veterinario (
    id INT AUTO_INCREMENT PRIMARY KEY,
    nome VARCHAR(80) NOT NULL,
    crmv VARCHAR(20) NOT NULL
);

CREATE TABLE pet (
    id INT AUTO_INCREMENT PRIMARY KEY,
    nome VARCHAR(60) NOT NULL,
    especie VARCHAR(40),
    tutor_id INT NOT NULL,
    FOREIGN KEY (tutor_id) REFERENCES tutor(id)
);

CREATE TABLE consulta (
    id INT AUTO_INCREMENT PRIMARY KEY,
    data_hora DATETIME NOT NULL,
    pet_id INT NOT NULL,
    veterinario_id INT NOT NULL,
    FOREIGN KEY (pet_id) REFERENCES pet(id),
    FOREIGN KEY (veterinario_id) REFERENCES veterinario(id)
);
```

### **ATENÇÃO — Ordem de criação importa**

Uma FOREIGN KEY exige que a tabela referenciada já exista. Por isso tutor e veterinario precisam ser criadas antes de pet e consulta — inverter essa ordem faz o CREATE TABLE falhar com um erro de referência desconhecida.

# O que é JDBC e o Papel do Driver

*A API que padroniza o acesso a bancos de dados a partir de uma aplicação Java*

JDBC significa Java Database Connectivity. É a forma padronizada que a linguagem Java oferece para se conectar a uma grande variedade de bancos de dados — relacionais, orientados a objetos e até alguns NoSQL — abstraindo as particularidades de cada um por trás de uma interface comum. Isso significa que o mesmo código de aplicação, em teoria, pode falar com bancos diferentes sem precisar ser reescrito.

É útil pensar no JDBC como um intermediário, posicionado entre a aplicação Java e a fonte de dados. Curiosamente, essa fonte de dados nem precisa ser um banco relacional tradicional — existem drivers JDBC para consultar planilhas e arquivos simples usando a própria sintaxe do SQL.

Só que, para conversar com uma fonte de dados específica, a aplicação precisa de um driver JDBC para ela. Um driver nada mais é do que uma biblioteca Java contendo as classes que implementam a API do JDBC para aquele banco específico. Os fabricantes costumam distribuir seus próprios drivers, normalmente como um arquivo JAR ou um módulo Java, que é importado no projeto.

## O que o driver nos permite fazer

- ▶ Conectar-se ao banco — cada fabricante pode ter um mecanismo próprio de autenticação e conexão.
- ▶ Executar instruções SQL — tanto DML (as operações de CRUD) quanto DDL.
- ▶ Executar stored procedures — enviando ao banco uma solicitação para rodar uma rotina já armazenada nele.
- ▶ Recuperar e processar resultados — um conjunto de linhas vindo de um SELECT, ou a contagem de linhas afetadas por um INSERT ou UPDATE.
- ▶ Lidar com exceções do banco de dados, que o JDBC expõe de forma consistente através de `SQLException`.

A API JDBC evolui em versões, e a atual é a JDBC 4.3 — mantendo compatibilidade retroativa com todos os métodos das versões anteriores. Vale garantir que o driver usado suporte, no mínimo, JDBC 4.0, que trouxe recursos importantes: carregamento automático de drivers, pool de conexões embutido, suporte a transações distribuídas, streaming de linhas e o padrão try-with-resources para fechar recursos automaticamente.

A API do JDBC está organizada em dois pacotes. O `java.sql` é o núcleo, com a maioria dos tipos usados no dia a dia. Já o `javax.sql` oferece alternativas voltadas a ambientes de servidor, para alguns dos tipos mais importantes.

| Finalidade                               | <code>java.sql</code>      | <code>javax.sql</code>  |
|------------------------------------------|----------------------------|-------------------------|
| Estabelecer conexão através de um driver | <code>DriverManager</code> | <code>DataSource</code> |
| Representar resultados de uma consulta   | <code>ResultSet</code>     | <code>RowSet</code>     |

**DICA — DataSource é o caminho recomendado**

DriverManager funciona bem para exemplos simples como os desta apostila, mas em aplicações reais o DataSource costuma ser preferido, por suportar pool de conexões e outras funcionalidades que o DriverManager não oferece nativamente.

# Conectando-se ao Banco com JDBC

*Montando a connection string certa para cada fabricante e abrindo a primeira conexão*

Toda conexão JDBC começa com uma connection string — uma URL que descreve, em um único texto, qual driver usar, onde o banco está e qual schema ou database acessar. A sintaxe geral é parecida entre fabricantes, mas cada um tem pequenas particularidades que é preciso respeitar.

| Fabricante | Formato da URL JDBC                                               |
|------------|-------------------------------------------------------------------|
| MySQL      | <code>jdbc:mysql://[host]:[porta]/[banco]</code>                  |
| PostgreSQL | <code>jdbc:postgresql://[host]:[porta]/[banco]</code>             |
| Oracle     | <code>jdbc:oracle:thin:@[host]:[porta]/[banco]</code>             |
| SQL Server | <code>jdbc:sqlserver://[host]:[porta];databaseName=[banco]</code> |
| SQLite     | <code>jdbc:sqlite:[arquivo].db</code>                             |

Repare nas diferenças sutis: o Oracle inclui um arroba antes do host, e o SQL Server usa ponto e vírgula antes do nome do banco, em vez da barra usada pelos demais. Para o schema pataviva criado no capítulo anterior, a URL do MySQL ficaria assim:

```
String url = "jdbc:mysql://localhost:3306/pataviva";
String usuario = "root";
String senha = "minhaSenhaSegura";

try (Connection conexao = DriverManager.getConnection(url, usuario, senha)) {
    System.out.println("Conectado à PataViva com sucesso!");
} catch (SQLException e) {
    e.printStackTrace();
}
```

O método `DriverManager.getConnection` devolve um objeto `Connection`, que representa a sessão ativa entre a aplicação e o banco — é a partir dele que instruções SQL serão criadas e executadas nos próximos capítulos. Repare que a conexão é aberta dentro de um bloco `try-with-resources`: como `Connection` implementa `AutoCloseable`, ela é fechada automaticamente ao final do bloco, mesmo se uma exceção for lançada no meio do caminho.

## ATENÇÃO — Conexões são um recurso caro

Abrir uma conexão com o banco envolve autenticação e negociação de rede — um custo real. Deixar de fechar conexões, seja por esquecimento ou por uma exceção não tratada, é uma das causas mais comuns de esgotamento de recursos em aplicações Java que usam JDBC diretamente.

# Statement e ResultSet: Executando e Lendo Consultas

*O jeito mais direto de mandar SQL para o banco e percorrer o que ele devolve*

Com uma `Connection` em mãos, o próximo passo é criar um `Statement` — o objeto responsável por enviar comandos SQL ao banco de dados. É a forma mais simples de executar SQL via JDBC: o texto do comando é montado como uma `String` comum, e enviado ao banco sem nenhuma etapa de pré-compilação.

Um `Statement` oferece três métodos principais de execução. `executeQuery` é usado para instruções `SELECT`, e devolve um `ResultSet` com as linhas encontradas. `executeUpdate` é usado para `INSERT`, `UPDATE` e `DELETE`, e devolve um `int` com o número de linhas afetadas. Já `execute` é o mais genérico dos três, útil quando não se sabe de antemão se a instrução vai devolver um resultado ou uma contagem.

```
String sql = "SELECT id, nome, especie FROM pet WHERE tutor_id = 1";

try (Statement stmt = conexao.createStatement();
     ResultSet rs = stmt.executeQuery(sql)) {

    while (rs.next()) {
        int id = rs.getInt("id");
        String nome = rs.getString("nome");
        String especie = rs.getString("especie");
        System.out.printf("Pet #%d: %s (%s)%n", id, nome, especie);
    }
}
```

O `ResultSet` funciona como um cursor: ele começa posicionado antes da primeira linha, e cada chamada a `next()` avança o cursor para a próxima linha disponível, devolvendo `false` quando não há mais nada a ler — por isso o padrão quase universal de percorrê-lo dentro de um laço `while`. Os métodos `getInt`, `getString`, `getDouble` e equivalentes leem o valor de uma coluna da linha atual, seja pelo nome da coluna ou pela sua posição numérica.

## ATENÇÃO — Statement e concatenação de texto do usuário não combinam

É tentador montar o SQL concatenando diretamente valores recebidos do usuário dentro da `String` do `Statement`. Essa prática é a porta de entrada mais comum para ataques de SQL Injection, tema do próximo capítulo — e é justamente o problema que o `PreparedStatement`, visto no capítulo 10, resolve de forma estrutural.

# SQL Injection e o Padrão ANSI SQL

*Como uma entrada de usuário mal tratada vira uma falha de segurança, e por que padronizar o SQL importa*

---

SQL Injection é uma vulnerabilidade séria, que ocorre quando um atacante consegue manipular os dados de entrada enviados a uma consulta SQL da aplicação, fazendo o banco executar algo bem diferente do que foi planejado. Os pontos de injeção mais comuns são formulários de login, campos de busca, parâmetros de URL usados para montar SQL dinâmico e, de forma geral, qualquer campo de entrada que acabe interagindo com o banco.

```
// Vulnerável: o nome digitado pelo tutor é concatenado direto no SQL
String nomeDigitado = entradaDoUsuario; // por exemplo: ' OR '1'='1
String sql = "SELECT * FROM tutor WHERE nome = '" + nomeDigitado + "'";
// a consulta final vira: SELECT * FROM tutor WHERE nome = '' OR '1'='1'
// que devolve TODOS os tutores, não apenas o pesquisado
```

## Como reduzir o risco

- ▶ Validar e higienizar toda entrada de usuário antes de usá-la.
- ▶ Aplicar o Princípio do Menor Privilégio, dando à conta do banco apenas as permissões estritamente necessárias.
- ▶ Implementar tratamento de erro adequado, sem expor nomes de tabelas ou detalhes do banco nas mensagens vistas pelo usuário.
- ▶ Usar PreparedStatement ou consultas parametrizadas sempre que possível — é o assunto do próximo capítulo, e a defesa mais eficaz contra esse tipo de ataque.

Mudando de assunto, mas ainda dentro do universo do SQL: cada fabricante de banco documenta suas próprias instruções, geralmente indicando o quanto essa implementação segue o padrão ANSI SQL — mantido pelo American National Standards Institute. Existe também uma versão equivalente da ISO, mantida pelo mesmo comitê técnico; por simplicidade, é comum chamar as duas coisas apenas de ANSI SQL.

Vale a pena buscar escrever código próximo do ANSI SQL sempre que possível, por algumas razões práticas: portabilidade entre bancos diferentes, já que o padrão é suportado, ao menos parcialmente, por todos os fabricantes relevantes; legibilidade, por ser uma sintaxe bem definida e fácil de entender entre times; manutenibilidade, já que é um padrão estável e pouco sujeito a mudanças; e, em alguns setores, exigências de conformidade regulatória.

Ainda assim, nem todo fabricante implementa o padrão da mesma forma. Um bom exemplo é a cláusula usada para limitar o número de linhas retornadas por uma consulta: o padrão ANSI define FETCH FIRST, mas o MySQL simplesmente não a suporta, preferindo sua própria cláusula LIMIT.



| Fabricante | TOP | LIMIT | FETCH FIRST |
|------------|-----|-------|-------------|
| SQL Server | Sim | Não   | Não         |
| MySQL      | Não | Sim   | Não         |
| PostgreSQL | Não | Sim   | Sim         |
| Oracle     | Não | Não   | Sim         |
| IBM Db2    | Não | Não   | Sim         |

#### **DICA — Comece pelo ANSI, ajuste ao vendor quando necessário**

Escrever primeiro pensando no padrão ANSI, e só recorrer a uma cláusula específica de um fabricante quando não houver alternativa, ajuda a minimizar o chamado vendor lock-in — a dependência excessiva das particularidades de um único banco.

# PreparedStatement: Consultas Parametrizadas

*Pré-compilando uma instrução SQL e substituindo valores por marcadores seguros*

Quando o driver JDBC envia uma instrução SQL ao servidor do banco, três coisas acontecem nos bastidores: a consulta é analisada (parseada), um plano de execução é montado, e o resultado — a instrução já compilada — fica em cache no servidor, pronto para ser reaproveitado sem precisar ser recompilado a cada nova execução.

Um PreparedStatement é justamente uma instrução SQL pré-compilada. A ideia lembra bastante a de compilar uma expressão regular com `Pattern.compile`: em ambos os casos, um texto precisa ser interpretado como uma operação, passando por análise de sintaxe e, opcionalmente, por otimizações — e esse processo tem um custo, que compensa pagar uma única vez quando a mesma instrução vai ser executada repetidamente.

A sintaxe usa um ponto de interrogação como marcador de posição para cada parâmetro — sem aspas ao redor dele, mesmo quando o valor final for um texto. Podem existir vários parâmetros na mesma instrução, de tipos diferentes; a forma de declarar o marcador é sempre a mesma, independentemente do tipo de dado que vai substituí-lo.

```
String sql = "INSERT INTO pet (nome, especie, tutor_id) VALUES (?, ?, ?)";

try (PreparedStatement stmt = conexao.prepareStatement(sql)) {
    stmt.setString(1, "Nina");
    stmt.setString(2, "Gato");
    stmt.setInt(3, 1);
    stmt.executeUpdate();
}
```

Os métodos `setString`, `setInt`, `setDouble` e equivalentes recebem a posição do marcador — começando em 1, não em 0 — e o valor a ser inserido ali. É justamente essa separação entre a estrutura fixa do SQL e os valores passados por parâmetro que impede um ataque de SQL Injection: o driver nunca interpreta o conteúdo do parâmetro como parte da sintaxe SQL, só como um valor literal.

## REGRA — Marcador sem aspas, sempre

Diferente de uma string literal em SQL, o marcador `?` nunca é envolvido por aspas simples, seja o parâmetro um número, um texto ou uma data. O tipo é resolvido pelo método `set` correspondente, não pela pontuação ao redor do marcador.

# Vantagens do PreparedStatement Sobre o Statement

*Seis motivos práticos para preferir instruções parametrizadas no dia a dia*

Depois de ver a sintaxe do PreparedStatement, vale consolidar por que ele costuma ser a escolha padrão em código JDBC de produção, em vez do Statement simples visto no capítulo 8.

| Vantagem                     | O que ela significa na prática                                                                                           |
|------------------------------|--------------------------------------------------------------------------------------------------------------------------|
| Pré-compilação               | O plano de execução é montado uma vez e reaproveitado, reduzindo o custo de execuções repetidas.                         |
| Consultas parametrizadas     | Os valores variáveis ficam isolados da estrutura do SQL, eliminando a superfície de ataque do SQL Injection.             |
| Reuso eficiente              | A mesma instância pode ser executada várias vezes, apenas trocando os valores dos parâmetros a cada execução.            |
| Conversão automática de tipo | Os métodos set cuidam da conversão entre o tipo Java e o tipo SQL correspondente.                                        |
| Legibilidade e manutenção    | O SQL fica visualmente limpo, sem concatenações espalhadas pelo código.                                                  |
| Segurança de tipos           | Erros de tipo tendem a aparecer em tempo de compilação ou de forma explícita, em vez de silenciosamente na string final. |

Um padrão bastante comum é reaproveitar o mesmo PreparedStatement para inserir vários registros relacionados, trocando apenas os valores dos parâmetros a cada iteração — por exemplo, cadastrar vários pets do mesmo tutor em sequência, sem recriar a instrução a cada linha.

```
String sql = "INSERT INTO pet (nome, especie, tutor_id) VALUES (?, ?, ?)";

try (PreparedStatement stmt = conexao.prepareStatement(sql)) {
    for (PetNovo p : petsDoTutor) {
        stmt.setString(1, p.nome());
        stmt.setString(2, p.especie());
        stmt.setInt(3, p.tutorId());
        stmt.executeUpdate();
    }
}
```

**DICA — Statement ainda tem seu lugar**

Para uma instrução DDL executada uma única vez — como criar uma tabela durante a inicialização de um script — um Statement simples continua sendo perfeitamente adequado. A escolha pelo PreparedStatement importa sobretudo quando há parâmetros vindos de fora do código, ou quando a mesma instrução roda repetidamente.

# Transações: AUTOCOMMIT, Commit e Rollback

*Garantindo que um grupo de operações relacionadas seja tratado como uma coisa só*

---

Por padrão, um objeto `Connection` nasce em modo AUTOCOMMIT: cada instrução executada é automaticamente persistida no banco assim que termina. Isso é conveniente para alterações pontuais e isoladas, mas perigoso quando um erro no meio de uma sequência de operações relacionadas pode deixar o banco em um estado inconsistente.

Por trás dos panos, alterações são primeiro gravadas em um local temporário — muitas vezes chamado de redo log ou journal — e só se tornam permanentes quando um commit é executado. Quando várias instruções precisam ser tratadas como uma única unidade atômica, a prática recomendada é desligar o AUTOCOMMIT antes de executá-las.

Em bancos de dados, uma transação é justamente essa série de uma ou mais operações tratadas como uma única unidade de trabalho: ou tudo é aplicado com sucesso, ou nada é. Se qualquer etapa falhar, a transação inteira é desfeita através de um rollback, e nenhuma das alterações daquele grupo chega a ser persistida. Em JDBC, uma transação é iniciada no momento em que o AUTOCOMMIT é desligado na conexão.

```

String inserirConsulta = "INSERT INTO consulta (data_hora, pet_id, veterinario_id)
VALUES (?, ?, ?)";

String atualizarAgenda = "UPDATE veterinario SET vagas_hoje = vagas_hoje - 1 WHERE id =
?";

try {
    conexao.setAutoCommit(false);

    try (PreparedStatement inserir = conexao.prepareStatement(inserirConsulta);
        PreparedStatement atualizar = conexao.prepareStatement(atualizarAgenda)) {

        inserir.setObject(1, dataHora);
        inserir.setInt(2, petId);
        inserir.setInt(3, veterinarioId);
        inserir.executeUpdate();

        atualizar.setInt(1, veterinarioId);
        atualizar.executeUpdate();

        conexao.commit();
    }
} catch (SQLException e) {
    conexao.rollback();

    throw e;
} finally {
    conexao.setAutoCommit(true);
}

```

### ATENÇÃO — Rollback no catch, sempre

Se qualquer instrução dentro do bloco de transação lançar uma exceção, é essencial chamar rollback antes de propagar o erro. Sem isso, mudanças parciais e inconsistentes correm o risco de ficar pendentes na conexão até o próximo commit acidental.

## Execução em Lote com executeBatch

*Agrupando várias instruções em uma única viagem até o banco de dados*

---

Quando muitas linhas precisam ser inseridas, atualizadas ou apagadas de uma vez, executar uma instrução JDBC por linha significa uma viagem de rede separada para cada uma — um custo que cresce rápido conforme o volume de dados aumenta. O método `executeBatch` resolve isso agrupando várias instruções e enviando todas juntas ao banco em uma única viagem.

```
String sql = "INSERT INTO consulta (data_hora, pet_id, veterinario_id) VALUES (?, ?, ?)";

try (PreparedStatement stmt = conexao.prepareStatement(sql)) {
    conexao.setAutoCommit(false);

    for (AgendamentoNovo a : agendamentosDoDia) {
        stmt.setObject(1, a.dataHora());
        stmt.setInt(2, a.petId());
        stmt.setInt(3, a.veterinarioId());
        stmt.addBatch();
    }

    int[] resultados = stmt.executeBatch();
    conexao.commit();

    System.out.println(resultados.length + " consultas agendadas em lote.");
} catch (SQLException e) {
    conexao.rollback();
    throw e;
} finally {
    conexao.setAutoCommit(true);
}
```

Cada chamada a `addBatch()` adiciona os valores atuais dos parâmetros à fila do lote, sem executar nada ainda. Só quando `executeBatch()` é chamado é que todas as instruções acumuladas são enviadas juntas, e o retorno é um array de inteiros com o número de linhas afetadas por cada instrução individual do lote.

Um detalhe importante: `executeBatch` não garante que todas as instruções do lote sejam executadas, caso uma delas falhe no meio do caminho — esse comportamento varia conforme o driver e sua configuração. Por isso, mesmo usando execução em lote, continua sendo essencial gerenciar a transação com `commit` e `rollback` ao redor dela, como no exemplo acima.

**REGRA — Batch por unidade de negócio, não por tabela inteira**

Ao inserir dados relacionados — como um pedido e seus itens, ou uma agenda inteira de consultas — é uma boa prática batchar apenas os registros de uma mesma unidade lógica por vez, com uma transação e um rollback dedicados a ela. Isso evita que uma falha em um único registro derrube o processamento de todos os demais.



## CAPÍTULO 14

# Update, Delete e Inserção de Registros Relacionados

*Modificando dados existentes sem quebrar a integridade referencial entre tabelas*

As instruções UPDATE e DELETE seguem a mesma lógica de parametrização já vista com INSERT: um PreparedStatement com marcadores de posição no lugar dos valores que mudam a cada execução.

```
String atualizar = "UPDATE consulta SET data_hora = ? WHERE id = ?";
try (PreparedStatement stmt = conexao.prepareStatement(atualizar)) {
    stmt.setObject(1, novaDataHora);
    stmt.setInt(2, consultaId);
    int linhasAfetadas = stmt.executeUpdate();
}

String remover = "DELETE FROM consulta WHERE id = ?";
try (PreparedStatement stmt = conexao.prepareStatement(remover)) {
    stmt.setInt(1, consultaId);
    stmt.executeUpdate();
}
```

O ponto de atenção real neste capítulo não está na sintaxe, mas na integridade referencial. As chaves estrangeiras definidas no capítulo 5 existem justamente para impedir que um registro pai seja apagado enquanto ainda existem registros filhos apontando para ele — tentar apagar um pet que ainda tem consultas associadas, por exemplo, faz o banco rejeitar o DELETE com um erro de violação de chave estrangeira, a menos que a tabela tenha sido configurada explicitamente para permitir exclusão em cascata.

Isso também afeta a ordem em que registros relacionados devem ser inseridos: como consulta referencia tanto pet quanto veterinario, o pet e o veterinário precisam já existir no banco antes que a consulta que os relaciona possa ser inserida. Um padrão comum é inserir o registro pai primeiro, capturar a chave gerada automaticamente com `getGeneratedKeys()`, e só então inserir os registros filhos usando essa chave.

```
String inserirPet = "INSERT INTO pet (nome, especie, tutor_id) VALUES (?, ?, ?)";

try (PreparedStatement stmt = conexao.prepareStatement(inserirPet,
Statement.RETURN_GENERATED_KEYS)) {

    stmt.setString(1, "Bidu");
    stmt.setString(2, "Cachorro");
    stmt.setInt(3, tutorId);
    stmt.executeUpdate();

    try (ResultSet chaves = stmt.getGeneratedKeys()) {
        if (chaves.next()) {
            int novoPetId = chaves.getInt(1);
            // agora novoPetId pode ser usado para inserir uma consulta, por exemplo
        }
    }
}
}
```

#### **DICA — getGeneratedKeys evita uma segunda consulta**

Sem esse recurso, seria preciso fazer um SELECT adicional só para descobrir o id que o banco acabou de gerar automaticamente para o novo registro — RETURN\_GENERATED\_KEYS entrega esse valor na própria resposta do INSERT.

# Stored Procedures e o CallableStatement

*Chamando rotinas que já vivem compiladas dentro do próprio banco de dados*

Trabalhar com bancos de dados via JDBC traz uma série de preocupações recorrentes: ajuste de performance, gerenciamento de transações, vazamento de recursos, mapeamento correto de tipos de dados, controle de concorrência, escalabilidade, testes e integração, dependência do fabricante, SQL Injection, e monitoramento e logging. Boa parte dessas preocupações pode ser endereçada movendo parte da lógica para o próprio servidor de banco de dados, na forma de rotinas escritas em SQL ou em uma linguagem procedural que integra bem com SQL.

Essas rotinas são chamadas, como categoria, de stored procedures — embora, tecnicamente, existam dois artefatos distintos: a procedure propriamente dita, e a function, que veremos no próximo capítulo. Uma stored procedure encapsula lógica SQL complexa e manipulação de dados em um módulo reutilizável, já compilado e armazenado como mais um objeto do banco — o que ajuda a melhorar performance, modularidade e segurança.

```
DELIMITER $$

CREATE PROCEDURE cancelar_consulta(IN idConsulta INT)

BEGIN

    DELETE FROM consulta WHERE id = idConsulta;

END $$

DELIMITER ;
```

Java executa e recupera resultados dessas rotinas armazenadas através de um tipo especial de instrução, o CallableStatement. Assim como um PreparedStatement, ele é criado a partir de uma string parametrizada — mas, nesse caso, o objeto referenciado já está compilado no servidor do banco. A relação lembra a de uma chamada de método comum: você passa dados como parâmetros, e pode ou não receber dados de volta.

```
String chamada = "{call cancelar_consulta(?)}";

try (CallableStatement stmt = conexao.prepareCall(chamada)) {

    stmt.setInt(1, consultaId);

    stmt.execute();

}
```

## DICA — A sintaxe de chave é padrão JDBC

A notação {call nome\_da\_procedure(?, ?)} é uma das JDBC Escape Sequences, que veremos com mais detalhe no capítulo 17 — ela permite chamar uma stored procedure de forma consistente, independentemente do banco de dados por trás da conexão.

## Parâmetros IN, OUT e INOUT

*Os três jeitos de trocar dados com uma stored procedure através do CallableStatement*

O MySQL define três tipos de parâmetro para suas procedures, e cada um determina a direção em que o dado pode fluir entre a aplicação Java e a rotina armazenada.

| Tipo  | Descrição                                                                                        | Lê? | Escreve? |
|-------|--------------------------------------------------------------------------------------------------|-----|----------|
| IN    | Usado para passar valores para a stored procedure. É o tipo padrão quando nenhum é especificado. | Sim | Não      |
| OUT   | Usado para devolver valores da procedure de volta para quem a chamou.                            | Não | Sim      |
| INOUT | Passa um valor para a procedure, que pode modificá-lo e devolvê-lo já alterado.                  | Sim | Sim      |

Do lado do Java, um parâmetro OUT ou INOUT precisa ser explicitamente registrado no CallableStatement antes da execução, informando o tipo SQL esperado — é assim que o driver sabe reservar espaço para o valor de retorno. Depois da execução, o valor é lido com o método get correspondente ao tipo registrado.

```
DELIMITER $$

CREATE PROCEDURE contar_consultas_do_dia(
    IN dataAlvo DATE,
    OUT total INT
)
BEGIN
    SELECT COUNT(*) INTO total
    FROM consulta
    WHERE DATE(data_hora) = dataAlvo;
END $$

DELIMITER ;
```

```
String chamada = "{call contar_consultas_do_dia(?, ?)}";

try (CallableStatement stmt = conexao.prepareCall(chamada)) {
    stmt.setDate(1, java.sql.Date.valueOf(dataAlvo));
    stmt.registerOutParameter(2, Types.INTEGER);

    stmt.execute();

    int totalDeConsultas = stmt.getInt(2);
    System.out.println("Consultas agendadas hoje: " + totalDeConsultas);
}
```

### **REGRA — Registre antes de executar**

Todo parâmetro OUT ou INOUT precisa passar por `registerOutParameter` antes da chamada a `execute()`. Esquecer esse passo é a causa mais comum de exceções ao trabalhar com `CallableStatement` pela primeira vez.

# Stored Functions e Escape Sequences do JDBC

*A outra metade das rotinas armazenadas, e a sintaxe que o JDBC usa para ficar portátil entre bancos*

Além das stored procedures, os RDBMS costumam suportar um segundo tipo de rotina armazenada: a stored function. Ambas ficam salvas no banco e são executadas como uma única unidade, mas têm propósitos diferentes. Uma stored function é desenhada para realizar um cálculo ou uma transformação de dado específica, e devolver um único valor como resultado.

| Característica                   | O que significa                                                                                   |
|----------------------------------|---------------------------------------------------------------------------------------------------|
| Devolve um valor                 | Toda stored function é esperada para retornar exatamente um valor, ao contrário de uma procedure. |
| É imutável                       | Não deveria ter efeitos colaterais — isto é, não deveria alterar dados no banco.                  |
| Pode ser usada em expressões SQL | É possível chamá-la diretamente dentro de um SELECT, uma cláusula WHERE ou uma condição de JOIN.  |

```
DELIMITER $$  
  
CREATE FUNCTION idade_do_pet(dataNascimento DATE) RETURNS INT  
DETERMINISTIC  
BEGIN  
    RETURN TIMESTAMPDIFF(YEAR, dataNascimento, CURDATE());  
END $$  
  
DELIMITER ;
```

| Stored Function      | Stored Procedure                      |
|----------------------|---------------------------------------|
| Validação de dado    | Modificação de dado                   |
| Conversão de dado    | ETL (extração, transformação e carga) |
| Expressões complexas | Lógica de negócio                     |
| Cálculos             | Processamento em lote                 |

É menos comum chamar uma stored function diretamente de um CallableStatement do que uma procedure, mas o recurso existe e vale conhecer. Há duas diferenças de sintaxe em relação à chamada de uma procedure: a string sempre começa com um marcador de posição reservado para o valor de retorno, e o MySQL só aceita parâmetros IN nas suas stored functions.

```
String chamada = "{? = call idade_do_pet(?)}";

try (CallableStatement stmt = conexao.prepareCall(chamada)) {
    stmt.registerOutParameter(1, Types.INTEGER);
    stmt.setDate(2, java.sql.Date.valueOf(dataNascimentoDoPet));
    stmt.execute();

    int idade = stmt.getInt(1);
    System.out.println("Idade estimada do pet: " + idade + " anos");
}
```

Essa sintaxe entre chaves — usada tanto para procedures quanto para functions — faz parte de um recurso maior chamado JDBC Escape Sequences: uma forma de executar operações específicas de banco de dados de maneira consistente e portátil entre diferentes SGBDs. Elas cobrem, entre outras coisas, literais de data, hora e timestamp, funções escalares numéricas e de texto, caracteres de escape em cláusulas LIKE, e chamadas a procedures e functions — tudo aquilo que naturalmente não é padronizado entre fabricantes.

#### **DICA — Duas ferramentas, um mesmo objetivo**

Stored procedures são a ferramenta certa para executar múltiplas operações, modificar dados e impor regras de negócio. Stored functions brilham em cálculos e transformações. As duas melhoram reuso, performance e encapsulamento — e, na prática, simplificam bastante o código JDBC do lado do Java.

# Introdução ao JPA e ao ORM

*Uma camada de abstração entre o código Java e o SQL que ele geraria manualmente*

---

Tudo o que vimos até aqui — Connection, Statement, PreparedStatement, CallableStatement — é código JDBC escrito diretamente. Existe uma alternativa que reduz bastante esse trabalho repetitivo: a Java Persistence API, ou JPA. Vale um esclarecimento logo de início: JPA é uma especificação, não uma implementação. O Java SE não traz nenhuma implementação padrão dela embutida — é preciso escolher um provider de JPA, como Hibernate, Spring Data JPA ou EclipseLink, para de fato usá-la.

Em 2019, a Java Persistence API mudou oficialmente de nome para Jakarta Persistence API, embora a sigla JPA continue sendo usada quase universalmente. É bastante provável que qualquer aplicação Java de porte médio ou grande que você venha a trabalhar utilize alguma forma de JPA, por isso vale a pena conhecer seus conceitos centrais, mesmo brevemente.

## Conceitos-chave da especificação

- ▶ Object-Relational Mapping (ORM) — a técnica central que dá nome à camada inteira.
- ▶ Entity — o objeto mapeado a partir de uma tabela do banco.
- ▶ EntityManager — o componente que gerencia o ciclo de vida dessas entidades.
- ▶ Persistence Context — a área de cache especial onde as entidades gerenciadas vivem.

É útil pensar no JPA como uma camada extra de funcionalidade, posicionada entre o código da aplicação e o código JDBC visto nos capítulos anteriores — o provider de JPA é quem, por baixo dos panos, continua gerando Connections, PreparedStatements e ResultSets.

## O que é Object-Relational Mapping

ORM é o método de mapear tabelas de um banco de dados — definidas por suas colunas — para classes Java com campos correlacionados, getters e setters. Um registro de uma tabela se torna, nesse modelo, uma instância de uma dessas classes. Frameworks de ORM costumam automatizar boa parte desse trabalho, com quatro benefícios recorrentes:

- ▶ Simplificam o código, reduzindo a necessidade de classes repetitivas de acesso a dados.
- ▶ Simplificam as operações de banco, escondendo a complexidade das consultas SQL.
- ▶ Aumentam a portabilidade do código entre diferentes sistemas de banco de dados.
- ▶ Incentivam uma abordagem mais estruturada e organizada de acesso a dados.



```

@Entity
@Table(name = "pet")
public class Pet {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Integer id;

    @Column(name = "nome")
    private String nome;

    @Column(name = "especie")
    private String especie;

    @ManyToOne
    @JoinColumn(name = "tutor_id")
    private Tutor tutor;

    // construtores, getters e setters omitidos por brevidade
}

```

A classe Pet é um exemplo relativamente simples de uma entidade anotada. Cada campo é mapeado a uma coluna correspondente da tabela, e a anotação @Id identifica qual campo representa a chave primária.

# Entity, EntityManager e Persistence Context

*Quem gerencia o ciclo de vida de um objeto mapeado, e onde ele vive enquanto isso*

O EntityManager é implementado pelo provider de JPA escolhido — Hibernate, no caso mais comum. Uma entidade pode existir em dois estados principais: gerenciada, quando está sob controle de um EntityManager, ou desanexada (detached), quando existe fora dele. Uma entidade desanexada pode voltar a ser gerenciada através de uma operação de merge, caso seja necessário persistir alguma alteração feita nela enquanto estava fora do EntityManager.

A interface EntityManager expõe métodos que se alinham diretamente com as quatro operações de CRUD já vistas no capítulo 3.

| Operação | Método  | Descrição                                                                                                                                                          |
|----------|---------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| create   | persist | Torna uma instância desanexada gerenciada e persistente.                                                                                                           |
| read     | find    | Busca uma entidade pela classe e chave primária informadas, devolvendo uma instância gerenciada.                                                                   |
| update   | merge   | Se a entidade já está gerenciada, as alterações são propagadas automaticamente ao banco no commit; se não está, merge a torna gerenciada e persiste as alterações. |
| delete   | remove  | Remove a instância do gerenciamento e executa a exclusão no banco, se a instância estiver gerenciada.                                                              |

```
EntityManagerFactory emf = Persistence.createEntityManagerFactory("pataviva-unit");
EntityManager em = emf.createEntityManager();

em.getTransaction().begin();

Pet novoPet = new Pet();
novoPet.setNome("Fred");
novoPet.setEspecie("Coelho");
novoPet.setTutor(em.find(Tutor.class, 1));

em.persist(novoPet); // create
em.getTransaction().commit();

Pet encontrado = em.find(Pet.class, novoPet.getId()); // read
System.out.println(encontrado.getNome());

em.close();
emf.close();
```

Repare como esse trecho substitui, sozinho, o equivalente a um PreparedStatement de INSERT e outro de SELECT do capítulo 10 — sem uma única string de SQL escrita à mão. É justamente esse tipo de redução de código repetitivo que costuma justificar a adoção do JPA em projetos maiores.

## O Persistence Context

O Persistence Context é a área especial, mantida internamente pelo EntityManager, onde as entidades gerenciadas efetivamente residem. Ele cumpre quatro papéis: acompanha o estado do ciclo de vida de cada entidade gerenciada; sincroniza alterações feitas nessas entidades com o banco de dados; garante identidade única para cada entidade dentro de uma mesma transação; e funciona como um cache, reduzindo idas desnecessárias ao banco ao manter entidades acessadas com frequência já disponíveis em memória.

### DICA — find() primeiro consulta a memória

Quando find() é chamado para um id que já está no Persistence Context, o EntityManager devolve a instância já em memória, sem sequer consultar o banco de dados novamente — mais um efeito prático do papel de cache do Persistence Context.

## Consultas com JPQL

*Uma linguagem de consulta orientada a objetos, pensada para trabalhar com entidades, não com tabelas*

Uma consulta a um banco de dados, na prática, quase sempre se resume a algum tipo de SELECT. Dentro do JPA, existem três formas de executar uma consulta, e todas elas continuam dependendo da existência de uma classe de entidade mapeada: a JPA Query, usando uma linguagem especial chamada JPQL; a CriteriaBuilder Query, uma forma mais programática de montar a consulta; e a Native Query, que recorre diretamente ao SQL do banco.

A Java Persistence Query Language, ou JPQL, é uma linguagem de consulta orientada a objetos. Ela foi desenhada especificamente para trabalhar com entidades em aplicações que usam JPA, permitindo consultar os dados armazenados no banco relacional sem que o código precise conhecer os detalhes de como esses dados estão estruturados ali. A sintaxe lembra bastante o SQL, mas as referências não são a tabelas e colunas — são a entidades e seus campos.

```
String jpql = "SELECT p FROM Pet p WHERE p.especie = :especie";

TypedQuery<Pet> query = em.createQuery(jpql, Pet.class);
query.setParameter("especie", "Gato");

List<Pet> gatos = query.getResultList();
gatos.forEach(p -> System.out.println(p.getNome()));
```

Repare em duas diferenças em relação ao SQL puro. Primeiro, Pet na cláusula FROM se refere à classe da entidade, com P maiúsculo, e não ao nome literal da tabela pet no banco. Segundo, o parâmetro é nomeado — :especie — em vez de posicional como no PreparedStatement, o que costuma deixar consultas com muitos parâmetros mais fáceis de ler e manter.

### REGRA — JPQL fala de entidades, SQL fala de tabelas

Uma consulta JPQL nunca deveria fazer referência direta ao nome de uma tabela ou coluna do banco — apenas aos nomes da classe de entidade e de seus campos, exatamente como declarados no código Java. É essa camada de indireção que mantém a consulta portátil entre diferentes bancos de dados.

## Joins em JPQL e o JPA Tuple

*Consultando dados que atravessam mais de uma entidade relacionada, sem trazer o objeto inteiro*

Assim como no SQL puro, é possível fazer joins em uma consulta JPQL — só que, em vez de uma condição explícita de igualdade entre colunas, o join costuma simplesmente percorrer o relacionamento já declarado nas próprias anotações da entidade, como o @ManyToOne visto no capítulo 18.

```
String jpql = "SELECT c.dataHora, p.nome, v.nome " +
    "FROM Consulta c " +
    "JOIN c.pet p " +
    "JOIN c.veterinario v " +
    "WHERE p.especie = :especie";

TypedQuery<Tuple> query = em.createQuery(jpql, Tuple.class);
query.setParameter("especie", "Cachorro");

List<Tuple> resultados = query.getResultList();
for (Tuple linha : resultados) {
    LocalDateTime dataHora = linha.get(0, LocalDateTime.class);
    String nomePet = linha.get(1, String.class);
    String nomeVet = linha.get(2, String.class);
    System.out.printf("%s - %s com %s\n", dataHora, nomePet, nomeVet);
}
```

Repare que a consulta acima não seleciona entidades inteiras — ela seleciona apenas três campos específicos, vindos de três entidades diferentes. Para esse cenário, o resultado não pode ser mapeado diretamente para uma única classe de entidade; é aí que entra o JPA Tuple.

Um Tuple é uma coleção ordenada de valores, retornada por uma consulta ao banco. Funciona como um contêiner leve de dados, mais flexível do que devolver entidades completas ou arrays de Object sem tipagem — é útil pensar nele como um mini-registro, carregando apenas os campos específicos de que a aplicação precisa naquele momento, acessados por posição ou por um alias definido na própria consulta.

### DICA — Tuple para leitura, Entity para escrita

Quando a consulta serve apenas para exibir informação — como um relatório que cruza pet, tutor e veterinário — um Tuple evita o custo de carregar entidades completas e gerenciadas. Quando o objetivo é depois alterar e persistir os dados, uma entidade gerenciada continua sendo o caminho certo.

# CriteriaBuilder: Consultas Programáticas e Type-Safe

*Montando uma consulta com objetos Java em vez de uma string de JPQL*

A interface `CriteriaBuilder` funciona como uma fábrica de componentes de consulta type-safe. Em vez de escrever a consulta como uma string — como acontece com JPQL — cada parte dela é construída como um objeto Java, verificado em tempo de compilação. Ela expõe métodos para criar cada peça da consulta, e cada método devolve uma peça específica: um `Predicate` representa uma condição de `WHERE`; uma `Expression` representa um valor computado; um `Order` representa um critério de ordenação; e uma `Selection` representa o que efetivamente será retornado pela consulta.

A `CriteriaQuery` é o objeto que vai reunindo essas peças — pense nela como um andaime: conforme métodos são chamados, a instância cresce e muda, incorporando cada nova cláusula. Já o `Root` é o tipo especial que serve de ponte entre a `CriteriaQuery` e a entidade sendo consultada — é a fonte de onde os dados da consulta partem. Como a maioria desses métodos devolve a própria instância de `CriteriaQuery`, é comum encadeá-los em sequência.

| Passo                                   | Método                                                                                          |
|-----------------------------------------|-------------------------------------------------------------------------------------------------|
| 1. Obter o <code>CriteriaBuilder</code> | <code>entityManager.getCriteriaBuilder()</code>                                                 |
| 2. Obter o <code>CriteriaQuery</code>   | <code>criteriaBuilder.createQuery()</code>                                                      |
| 3. Definir a entidade raiz da consulta  | <code>criteriaQuery.from(Entidade.class)</code>                                                 |
| 4. Selecionar os dados desejados        | <code>criteriaQuery.select(root)</code> ou <code>.multiselect(...)</code>                       |
| 5. Adicionar predicados, ordenação etc. | métodos do <code>criteriaBuilder</code> , passados como argumento                               |
| 6. Obter a consulta executável          | <code>entityManager.createQuery(criteriaQuery)</code>                                           |
| 7. Executar e obter o resultado         | <code>getResultList()</code> , <code>getResultStream()</code> ou <code>getSingleResult()</code> |

```
CriteriaBuilder cb = em.getCriteriaBuilder();
CriteriaQuery cq = cb.createTupleQuery();

Root<Consulta> consulta = cq.from(Consulta.class);
Join<Consulta, Pet> pet = consulta.join("pet");
Join<Consulta, Veterinario> vet = consulta.join("veterinario");

cq.multiselect(
    consulta.get("dataHora"),
    pet.get("nome"),
    vet.get("nome"))
    .where(cb.equal(pet.get("especie"), "Cachorro"));

List<Tuple> resultados = em.createQuery(cq).getResultList();
```

O método `join`, chamado diretamente sobre o `Root`, cria a mesma junção entre entidades que fizemos com a palavra-chave `JOIN` em JPQL no capítulo anterior — só que como um objeto Java encadeável. E quando a seleção envolve mais de um campo, como neste exemplo, `multiselect` substitui o `select` usado para uma entidade inteira.

#### **DICA — CriteriaBuilder brilha em consultas dinâmicas**

A grande vantagem prática do `CriteriaBuilder` aparece quando a consulta precisa ser montada dinamicamente em tempo de execução — por exemplo, adicionando um predicate de filtro por espécie apenas se o usuário tiver informado esse filtro. Isso é bem mais natural de expressar com objetos Java do que concatenando pedaços de uma string JPQL.

# Native Queries e o Panorama Final das Consultas JPA

*Recorrendo ao SQL puro quando necessário, e comparando as três formas de consultar com JPA*

Nem toda consulta se encaixa bem em JPQL ou CriteriaBuilder — otimizações específicas de um banco, funções proprietárias, ou uma consulta legada já pronta em SQL às vezes são mais simples de reaproveitar diretamente. Para esses casos, o JPA oferece a Native Query, que executa SQL de verdade contra o banco, através do mesmo EntityManager.

```
String sql = "SELECT c.data_hora, p.nome, v.nome " +
    "FROM consulta c " +
    "JOIN pet p ON p.id = c.pet_id " +
    "JOIN veterinario v ON v.id = c.veterinario_id " +
    "WHERE p.especie = ?";

Query nativa = em.createNativeQuery(sql);
nativa.setParameter(1, "Gato");

List<Object[]> resultados = nativa.getResultList();
for (Object[] linha : resultados) {
    System.out.println(linha[0] + " - " + linha[1] + " com " + linha[2]);
}
```

Note que, diferente de JPQL, uma Native Query volta a referenciar nomes reais de tabelas e colunas do banco — a portabilidade entre fabricantes fica em segundo plano, em troca de controle total sobre a sintaxe SQL. Também é comum que o resultado venha como um array de Object por linha, sem a tipagem que um Tuple oferece, a menos que a consulta seja mapeada explicitamente para uma entidade ou uma classe de resultado.

## As três formas de consulta, lado a lado



| Tipo                  | Descrição                                                                | Vantagens                                                                                                  |
|-----------------------|--------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------|
| Native Query          | Executa SQL nativo diretamente                                           | Controle total sobre a sintaxe; otimização específica do banco; acesso a recursos exclusivos do fabricante |
| CriteriaBuilder Query | Monta a consulta programaticamente com objetos Java                      | Type-safe; verificação em tempo de compilação; boa para consultas montadas dinamicamente                   |
| JPQL Query            | Expressa a consulta em uma sintaxe parecida com SQL, voltada a entidades | Portável entre providers de JPA; esconde detalhes de implementação; intuitiva para quem já conhece SQL     |

Não existe uma escolha universalmente certa entre as três — a decisão depende do quanto a consulta precisa ser dinâmica, do quanto ela se beneficia de recursos específicos do banco, e de quão confortável o time está lendo cada sintaxe. Consultas fixas e simples tendem a favorecer JPQL pela legibilidade; consultas que mudam de forma em tempo de execução tendem a favorecer CriteriaBuilder; e consultas que dependem de algum recurso muito específico do banco tendem a favorecer uma Native Query.

#### **REGRA — Prefira a opção mais portátil que resolve o problema**

Como regra geral, vale começar por JPQL, subir para CriteriaBuilder quando a consulta precisar ser montada dinamicamente, e só recorrer a uma Native Query quando nenhuma das duas opções anteriores conseguir expressar o que é necessário.

## **Fim de apostila**

Este material percorreu o acesso a bancos de dados em Java de ponta a ponta: os fundamentos de um banco relacional e do SQL, a conexão via JDBC com Statement, PreparedStatement e CallableStatement, o cuidado com segurança e transações, e a camada de abstração que o JPA oferece sobre tudo isso — da entidade mapeada até as três formas de consultar dados através dela.

Esses conceitos sustentam qualquer aplicação Java que precise persistir e consultar dados de forma confiável — de um sistema de agendamento simples, como a PataViva usada em exemplos ao longo desta apostila, a sistemas corporativos de maior porte.